



Classification of facial expressions using custom CNNs and pre-trained models

Roberta Angioni, 60/73/65312

Mattia Cani, 60/73/65306

The task	3
Theoretical background	3
Classification	3
Convolutional Neural Network	4
ResNet	5
EfficientNet	5
MobileNet	6
Cross-Entropy Loss	7
Focal Loss	7
Optimization: ADAM	8
Regularization	9
Dropout	9
Weight decay	9
The dataset	10
Preprocessing	10
Training	11
Training the CNNs	11
Fine-tuning the pre-trained models	12
Testing	13
Testing the CNNs	13
Testing the pre-trained models	14
Evaluation	15
Metrics	16
CNN - 2 convolutional layers	16
CNN - 3 convolutional layers	17
CNN - 4 convolutional layers	18
EfficientNet	19
MobileNet	20
ResNet - training for 5 epochs	21
ResNet - training for 10 epochs	22
Confusion matrices	23
Comment on the results	24

The task

The task addressed in this project involves the classification of images from the FER2013 dataset (<https://www.kaggle.com/datasets/msambare/fer2013>) based on the facial expressions they represent. Specifically, three custom convolutional neural networks trained from scratch and the ResNet18, EfficientNet-B1 and MobileNet V2 models are used. So, transfer learning is leveraged through these three pre-trained models.

All the models are trained or fine-tuned for 5 epochs. However, a case in which ResNet is fine-tuned for 10 epochs is also taken into consideration, to verify its behavior as the epochs increase and justify the choice to stop at 5 in the other tests.

This classification exercise is approached in two different versions: binary, considering only the classes related to happy and sad emotions, and multiclass, where the classes related to anger and fear are also included. Both versions are subjected to the same tests conducted using all three models, which are evaluated according to the following metrics: accuracy, recall, precision and F1 score, considered both on individual classes and in general. All tests are done twice, in order to compare the performances obtained with two different loss functions: cross-entropy loss and focal loss.

The programming language used is Python.

Theoretical background

In this paragraph, all the concepts that relate to the project and techniques applied are explained in detail.

Classification

Image classification is a task where the goal is to automatically assign a label to an image based on its visual content. In our case, the process includes the following steps:

- Preprocessing of a dataset of images, that includes applying transformations and data augmentation techniques in order to correct some characteristics of the dataset that can negatively affect the labeling task.
- Feature extraction done by a deep learning model, such as our CNNs and pre-trained models, that extract relevant features from the image, such as edges, textures and facial landmarks.
- Classification, done by the last layer of the model. A fully connected layer with a softmax activation assigns probabilities to each class, and the image is classified based on the highest probability of belonging to a certain class according to the model's predictions.

In order to ensure a better choice of parameters, optimization and regularization techniques used by the models and spot possible issues, their performance is tested on a validation set before being tested and evaluated on the actual test set.

Convolutional Neural Network

CNNs are neural network architectures designed to process data with a grid-like topology, such as images. They utilize convolutional layers to automatically and adaptively learn spatial hierarchies of features from input data. Key components of CNNs include:

- **Convolutional Layers:** Extract features by applying filters (kernels) to input data.
- **Pooling Layers:** Downsample feature maps to reduce dimensionality and computational cost while preserving key information.
- **Fully Connected Layers:** Combine learned features for classification tasks.
- **Activation Functions:** Introduce non-linearity to enhance learning capacity.

In the context of this project, the CNN used is defined using PyTorch's `nn.Module` class and follows the typical structure:

- **Three Convolutional Layers:**
 - The first one takes the input image and applies 32 filters of size 3x3. Padding is applied to maintain the spatial dimensions of the output. A batch normalization layer follows the convolution to stabilize training and speed up convergence.
 - The second one receives the output from the first layer and applies 64 filters of size 3x3. It also uses batch normalization for further refinement.
 - The last one receives the output from the second layer and applies 128 filters of size 3x3. As in the previous layers, batch normalization is applied.
- **Pooling Layer:** after the convolutional block, a max-pooling operation with a kernel size of 2x2 is applied. This reduces the spatial dimensions of the feature map, effectively downsampling the data and reducing the number of parameters and computational cost.
- **Dropout Layer:** to mitigate overfitting, a dropout layer is included after the convolutional ones. Dropout randomly sets a fraction (50% in this case) of the input units to zero during training, which helps prevent the model from becoming too reliant on certain features.
- **Two Fully Connected Layers:**

- The first one has 512 units, and it takes the flattened output from the convolutional layers. A ReLU activation function is applied to introduce non-linearity and help learn complex patterns.
- The second fully connected layer outputs the final predictions, so it produces a vector of raw scores for each class. The number of units in this layer corresponds to the number of classes.

In addition to the original CNN, two additional architecturally distinct CNNs have been introduced. Variant 2 features more convolutional layers than the original CNN but has a smaller Fully Connected (FC) layer. Specifically, an extra convolutional layer has been added (three instead of two), the number of neurons in the Fully Connected layer has been reduced (128 instead of 256 or more), and the convolutional layers include more filters (up to 256).

Variant 3, on the other hand, has fewer convolutional layers but a larger Fully Connected layer. Its structure is characterized by only two convolutional layers instead of three or more, while the Fully Connected layer contains more neurons (512 instead of 128-256). This third variant is designed to be simpler and more efficient, making it particularly well-suited for smaller datasets.

ResNet

ResNet18 is part of the ResNet family (Residual Networks) and is a deep pre-trained convolutional neural network that efficiently addresses vanishing gradient issues in deep learning.

- **Residual Blocks:** ResNet uses skip connections to allow gradients to flow directly through the network, enabling deeper architectures without performance degradation.
- **Pre-trained Weights:** the model is initially trained on a large dataset and fine-tuned on the dataset used for emotion recognition.

ResNet18 has 18 layers in total. In particular, the structure is roughly composed as it follows:

1. **Initial Convolution and Pooling:** a first convolutional layer with max pooling.
2. **Residual Blocks:** four blocks with two 3x3 convolutions each and increasing filters.
3. **Global Average Pooling**
4. **Fully Connected Layer:** computes raw scores for classification tasks (by default, it works with 1000 output classes)
5. **Softmax activation:** transforms raw scores into probabilities

EfficientNet

EfficientNet-B1 is part of the EfficientNet family, a series of convolutional neural networks designed to achieve high accuracy while maintaining computational efficiency through a balanced scaling approach.

- **Optimized scaling:** it uses compound scaling, a method that proportionally balances depth (number of layers), width (number of channels per layer), and input resolution. Unlike traditional models that scale only one dimension (like making the network deeper or wider), compound scaling improves performance without excessive computational cost by optimizing all three aspects simultaneously.
- **MBConv Blocks:** it uses Mobile Inverted Bottleneck Convolution blocks, derived from depthwise separable convolutions. They improve efficiency by using pointwise convolutions to reduce dimensionality, followed by depthwise (separate per-channel) convolutions for feature extraction.
- Includes **skip connections** similar to ResNet.
- **Pre-trained Weights:** in the same way as ResNet, the model is pre-trained on a large general-purpose dataset and can be fine-tuned for specific tasks.

The structure is composed as it follows:

1. **Initial Layer:** convolutional layer.
2. **MBConv Blocks:** 16 blocks with increasing filters (up to 320 filters are applied in the last block). Kernel size could be 3x3 or 5x5 based on the block and it's the same with the stride, that can be 1 or 2.
3. Final **pointwise convolutional layer** with 1280 filters.
4. **Global Average Pooling**
5. **Fully Connected Layer:** as for ResNet, it computes raw scores for classification tasks with the same number of output classes by default.
6. **Softmax activation** to obtain probabilities.

MobileNet

MobileNetV2 is part of the MobileNet family, which is composed of convolutional neural networks designed for deep learning on mobile and edge devices. It achieves a balance between high accuracy and low computational cost by using optimized architectural components.

- **Optimized Efficiency:** it is designed with a lightweight structure, reducing the number of parameters while maintaining performance. It introduces linear bottleneck layers and inverted residual connections, which improve information flow and reduce memory usage.
- **MBConv Blocks:** the architecture is based on Mobile Inverted Bottleneck Convolution blocks, similar to EfficientNet.
- **Pre-trained Weights:** like ResNet and EfficientNet, MobileNetV2 can be pre-trained on large general-purpose datasets and later fine-tuned for specific tasks.

The model has the following structure:

1. **Initial Layer:** convolutional layer.
2. **MBConv Blocks:** 17 blocks, progressively increasing the number of filters (up to 320 filters in the last stage). Expansion factors are applied to increase feature

representation before depthwise convolutions. Kernel size is 3x3 and the stride can be 1 or 2 depending on the stage.

3. **Pointwise convolution** with 1280 filters.
4. **Global average pooling**
5. **Fully Connected Layer:** computes raw scores for classification tasks.
6. **Softmax Activation** to produce class probabilities.

Cross-Entropy Loss

It is a widely used loss function for classification tasks, particularly when dealing with multi-class problems. It measures the difference between the predicted probability distribution (output of the model) and the true probability distribution (ground truth labels). In particular, this loss is calculated as the negative logarithm of the predicted probability assigned to the correct class. This encourages the model to assign higher probabilities to the correct class during training.

$$H(P^* | P) = - \sum_i \underbrace{P^*(i)}_{\text{TRUE CLASS DISTRIBUTION}} \log \underbrace{P(i)}_{\text{PREDICTED CLASS DISTRIBUTION}}$$

Focal Loss

It is a loss function designed to address the class imbalance problem in classification tasks, especially in scenarios where some classes are underrepresented. Actually, the focal loss is a modification of the standard cross-entropy loss that down-weights the loss for well-classified examples and focuses more on hard-to-classify examples.

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t).$$

where p_t is the probability for the correct class, α is a class balancing factor, γ increases emphasis on the classes that the model finds difficult.

So, focal loss ensures that the model focuses more on improving the classification of hard examples while minimizing the effect of well-classified ones.

Optimization: ADAM

The choice of the Adam optimizer in our case is driven by the fact that emotion recognition requires the analysis of complex information from visual data, and Adam is designed to perform well in scenarios with many variables and deep models. It can converge faster compared to other methods and adapt better to the peculiarities of the dataset. Furthermore, the dataset used in our case consists of low-resolution images, which are likely to exhibit some variability in facial expressions and Adam is capable of reducing large gradient oscillations, ensuring that the model proceeds in a controlled manner without skipping optimal solutions.

In detail, Adam is an optimizer that combines the features of two methods, Momentum and RMSprop, to achieve more efficient weight updates during the training of a model. This is how it works:

- Momentum focuses on using an exponential moving average of the previous gradient, which helps to accelerate in the important directions of the gradient. This reduces oscillations and stabilizes the optimization process, ensuring the optimizer continues to move in directions where the gradients have historically been strong.
- RMSprop, on the other hand, normalizes the gradient by dividing it by an exponential moving average of the squared gradients. This helps to avoid taking too large steps, which could lead to instability. By normalizing the gradient based on its variance, RMSprop ensures the step size is adapted to the scale of the gradient in each direction.

Adam combines these two techniques in various steps:

1. Calculate the gradients for each model parameter.
2. Estimate the first moment (similar to Momentum), which is an exponential moving average of the gradients. This helps to smooth out oscillations and accelerates movement in stable directions.
3. Estimate the second moment (similar to RMSprop), which is the exponential moving average of the squared gradients. This provides an estimate of the variance, helping to avoid too-large steps.
4. Apply an initial correction to the moments to prevent instability at the beginning of the optimization process (when the moment estimates are still small).
5. Divide the first moment by the square root of the second moment, with a small bias term added to avoid division by zero. This adaptive step allows the optimizer to dynamically adjust the learning rate for each parameter.
6. Make the weight update by subtracting the calculated value from the old weight, to obtain the new weight.

Regularization

Dropout

Dropout is a regularization technique used in deep learning models like CNNs to prevent overfitting by randomly deactivating a fraction of neurons during training. When a neuron is dropped out, it does not participate in the forward or backward pass, meaning its contribution to the network is temporarily removed. From a practical point of view, this is done by multiplying by zero a certain number of entries chosen randomly in the feature maps.

When dropout is set to 0.3 like in our CNNs, it means that 30% of the neurons in a given layer are randomly deactivated during each training iteration. However, during inference, all neurons are used and their outputs are scaled appropriately to compensate for the missing neurons during training.

By forcing the network to rely on different subsets of neurons at each iteration (different subsets of neurons are active in each iteration, so it makes training resemble the creation and training of multiple smaller networks), it prevents the model from memorizing specific training examples and encourages generalization to new data. Plus, the network learns to distribute the importance of features across different neurons rather than relying too much on a few dominant ones.

By using dropout with a rate of 0.3, we regularize the model without excessively reducing capacity.

Weight decay

Weight decay, also known as L2 Regularization, is another technique used in deep learning to prevent overfitting by penalizing excessively large weights and, in our case, it is used during the fine-tuning of pre-trained models. The main idea is simple: if the weights of a neural network become too large, the model may overfit the training data, so it encourages the network to keep the weights small, making it more robust. This is done by adding a penalty term to the loss function that is proportional to the sum of the squared weights.

Mathematically, during Stochastic Gradient Descent (SGD) or other optimization methods, Weight Decay modifies the weight update rule by introducing a penalty term λW , where:

- W represents the weights of the network.
- λ is a small regularization coefficient that controls how much the weights are penalized.

At each update step, the optimizer subtracts this small value from the weights, effectively pulling them closer to zero over time. This prevents the model from relying too much on large weight values, leading to better generalization, and reduces extreme fluctuations in weight updates, making training more stable.

In our case, weight decay is particularly useful because pre-trained neural networks have many parameters, and Weight Decay ensures that no single weight dominates the decision-making process, improving robustness and avoiding overfitting issues.

The dataset

Before explaining all the preprocessing techniques applied on the dataset, it is appropriate to give an overview about its original version: FER-2013 is a widely used benchmark for facial expression recognition and it contains 35,887 grayscale images with a resolution of 48x48 pixels, collected from various web sources using web scraping techniques. Each image depicts a face displaying one of the following seven emotions:

1. Anger (*angry*)
2. Disgust (*disgust*)
3. Fear (*fear*)
4. Happiness (*happy*)
5. Sadness (*sad*)
6. Surprise (*surprise*)
7. Neutral (*neutral*)

The dataset is divided into a training set and a test set for each class, with the majority of data allocated for training purposes. It is one of the most commonly used datasets in the field of emotion recognition due to its accessibility and the diversity of images, which include faces with different angles, lighting conditions, and demographic characteristics.

However, this dataset doesn't come without challenges: images have a very low resolution, which can make it difficult for models to learn subtle facial expression features, and the class distribution is far from being balanced, with some emotions being more represented than others. For example, the images depicting disgust are just over 400 in the training set, while other classes have 4000 images on average with an exception for the *happy* class, which has just over 7000 images and is by far the most represented class.

Preprocessing

The first step carried out for preprocessing the dataset was to choose which classes to use for the classification tasks. Having to choose two classes for the binary task, the choice fell on the two classes which are both the most represented and the least at risk of having overlapping features between them: *happy* and *sad*.

The choice to focus on the most represented classes is driven by the desire to avoid excessive data augmentation, which could lead to training that pushes the models to overly focus on specific image features despite the use of augmentations such as translations and rotations, ultimately failing to generalize in the testing phase. The *disgust* class also has a high probability of being well separable from the *happy* class, but its training set would have

needed to become 18 times larger through data augmentation to reach the same number of images as the *happy* class, which would likely lead the models to overfit.

The choice of classes to use for the multiclass task followed the same criterion, so the following classes have been chosen:

1. angry
2. fear
3. happy
4. sad

The reason why the multiclass version of the dataset used includes only 4 classes – instead of the original 7 – lies in the attempt to reduce the computational load in favor of faster performance and more accessible testing. To ensure a balanced dataset, the number of images per class used during the training phases was augmented to 8000 by applying random rotations up to 30 degrees, random horizontal flips and random cropping and resizing later – all the images have the same size after the preprocessing techniques are applied. To face the resolution issue, a median filter for denoising is applied to all the images.

Additional preprocessing techniques used include resizing the images – 48x48 for use with the CNN and 224x224 in the case of pre-trained models – and their normalization.

Training

Training the CNNs

The `train_model_CNN` function is designed to train the CNNs while tracking and saving the training and validation performance across multiple epochs. The function handles the full training process, including the forward pass, loss calculation, backpropagation, and parameter updates. Additionally, it computes accuracy metrics for both the training and validation sets and saves the results in a CSV file for later analysis.

The training process is carried out by a train loop, so for each epoch the model is set in training mode, then the following main steps are performed for each batch:

1. The optimizer gradients are reset.
2. The model makes predictions by passing the images through the forward pass, producing the raw output scores.

3. The loss is computed by comparing the model's output with the ground truth labels using the specified loss function.
4. Backpropagation is performed and calculates the gradients.
5. The optimizer updates the model's parameters based on the gradients calculated in the previous step.
6. The running loss is accumulated to track how well the model is performing during the training phase.

After completing the training, the `validate_model` function is called to evaluate the model's performance on the validation dataset, so this function computes the validation accuracy.

```
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad() # Reset gradients
        outputs = model(images) # Pass through the model
        loss = criterion(outputs, labels) # Calculate loss
        loss.backward() # Calculate gradients
        optimizer.step() # Update weights

        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    train_loss = running_loss / len(train_loader)
    train_accuracy = 100 * correct / total

    # Validate
    val_accuracy = validate_model(model, val_loader, device)

    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {train_loss:.4f}, "
          f"Accuracy: {train_accuracy:.2f}%, Validation Accuracy: {val_accuracy:.2f}%")
```

training phase

Fine-tuning the pre-trained models

In the same way, the `train_model_TL` function is responsible for training the pre-trained models. It is trivial that the training process in this case is not made from scratch, only the last layers are fine-tuned for our specific task.

The function starts by freezing the parameters of all layers in the chosen model. This is done to retain the pre-trained feature extraction capabilities of the earlier layers while preventing them from being updated during training, then the last two layers are released to adjust them to learn the task-specific features. The training loop performs exactly the same steps that are performed during the CNN training, and at the end of the computation the results are saved in a CSV file. In every model, only the last few layers are unfreezed in such a way to obtain a functional equity in how much of the model is trained again for emotion recognition.

```
def freeze_efficientnet_layers(model):
    # Freeze all layers
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze the last few layers
    for param in model.features[-1].parameters():
        param.requires_grad = True

    for param in model.classifier.parameters():
        param.requires_grad = True
```

```
def freeze_mobilenet_layers(model):
    # Freeze all layers
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze the last few layers
    for param in model.features[-3:].parameters():
        param.requires_grad = True

    for param in model.classifier.parameters():
        param.requires_grad = True
```

```
def freeze_resnet_layers(model):
    # Freeze all layers
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze the last few layers
    for param in model.layer4.parameters():
        param.requires_grad = True

    for param in model.fc.parameters():
        param.requires_grad = True
```

These functions are called at the start to freeze and adjust layers.

Testing

Testing the CNNs

For convenience reasons, the testing part is carried out by the `train_and_test_CNN` that, as the name suggests, handles the entire workflow for training and validating – by calling the functions previously explained – and for testing our CNN models. Here's a high-level breakdown of its functionality:

- A series of transformations for the input images, including resizing, tensor conversion and normalization are performed to all the dataset.
- The function splits the training dataset into a training and validation set (80% for training, 20% for validation).
- Based on the classification mode (binary or multiclass), the function initializes a CNN model with the appropriate number of output classes.
- Based on the parameters with which it is called, the function chooses the loss function and the optimizer to use.
- The model is then trained and saved in a .pth file.
- The saved model is loaded and passed to the `evaluate_model` function that will perform the actual test on the test dataset and save the results in a CSV file.

The `evaluate_model` function, for each batch of test images, performs a forward pass to obtain predictions, updates the total and class-specific correct predictions and populates the confusion matrix.

```

with torch.no_grad():
    for images, labels in test_loader:

        images, labels = images.to(device), labels.to(device)

        # Perform a forward pass
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        # Update total and correct predictions
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        # confusion matrix update
        for i in range(len(labels)):
            true_label = labels[i].item()
            pred_label = predicted[i].item()
            class_total[true_label] += 1
            if true_label == pred_label:
                class_correct[true_label] += 1
            confusion_matrix[true_label][pred_label] += 1

```

evaluate_model's main loop. The confusion matrix is then used to calculate metrics

Testing the pre-trained models

The train_and_test_TL function is designed to train and evaluate our pre-trained models and, as for the correspondent function for the custom CNN, it supports both binary and multiclass classification and integrates the entire pipeline, including preprocessing, training, validation, and testing that are carried out in a very similar way as in the correspondent function for the CNNs. The function saves the trained model in a .pth file and the results in a CSV file.

Before the training of the model, based on which parameter is passed to the function, the corresponding pre-trained model is loaded with its default weights, the final fully connected layer is adjusted to match the number of classes based on the mode.

```

if model_name == "resnet":
    model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    num_features = model.fc.in_features
elif model_name == "efficientnet":
    model = models.efficientnet_b1(weights=models.EfficientNet_B1_Weights.DEFAULT)
    num_features = model.classifier[1].in_features
elif model_name == "mobilenet":
    model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)
    num_features = model.classifier[1].in_features

if mode == "binary":
    if model_name == "resnet":
        model.fc = nn.Linear(num_features, 2)
    elif model_name == "efficientnet":
        model.classifier[1] = nn.Linear(num_features, 2)
    elif model_name == "mobilenet":
        model.classifier[1] = nn.Linear(num_features, 2)
    weights = torch.tensor([1.0, 1.5], dtype=torch.float32)
elif mode == "multiclass":
    if model_name == "resnet":
        model.fc = nn.Linear(num_features, 4)
    elif model_name == "efficientnet":
        model.classifier[1] = nn.Linear(num_features, 4)
    elif model_name == "mobilenet":
        model.classifier[1] = nn.Linear(num_features, 4)
    weights = torch.tensor([2.0, 1.5, 1.0, 1.3], dtype=torch.float32)

```

Load and adjust model.

Evaluation

This paragraph will present and discuss the results obtained during the testing phase. The following bullet list shows how the classes have been labeled:

- Binary task
 - 0 = happy
 - 1 = sad
- Multiclass task
 - 0 = angry
 - 1 = fear
 - 2 = happy
 - 3 = sad

Metrics

CNN - 2 convolutional layers

test_binario_crossentropy

Metric	Value
Test Accuracy	80.23833167825224
Macro Precision	80.39424189814814
Macro Recall	81.339272879488
Macro F1 Score	80.11491326652626
Accuracy Class 0	75.0281848928974
Accuracy Class 1	87.65036086607859
Precision Class 0	89.62962962962962
Precision Class 1	71.15885416666666
Recall Class 0	75.02818489289741
Recall Class 1	87.65036086607859
F1 Score Class 0	81.681497391838
F1 Score Class 1	78.54832914121451

test_binario_focal

Metric	Value
Test Accuracy	80.23833167825224
Macro Precision	80.22397839414728
Macro Recall	81.17251414669164
Macro F1 Score	80.0839251807014
Accuracy Class 0	75.8173618940248
Accuracy Class 1	86.52766639935847
Precision Class 0	88.89623265036352
Precision Class 1	71.55172413793103
Recall Class 0	75.8173618940248
Recall Class 1	86.52766639935847
F1 Score Class 0	81.83754183145726
F1 Score Class 1	78.33030852994554

test_multiclasse_crossentropy

Metric	Value	
Test Accuracy	45.95242854287427	
Macro Precision	56.24475595078617	
Macro Recall	44.02036060426506	
Macro F1 Score	42.5414722165843	
Top-2 Accuracy	71.11732960223866	
Accuracy Class 0	16.701461377870565	
Accuracy Class 1	33.69140625	
Accuracy Class 2	43.010146561443065	
Accuracy Class 3	82.6784282277466	
Precision Class 0	54.23728813559322	
Precision Class 1	46.1847389558233	
Precision Class 2	91.59663865546219	
Precision Class 3	32.96035805626599	
Recall Class 0	16.701461377870565	
Recall Class 1	33.69140625	
Recall Class 2	43.01014656144307	
Recall Class 3	82.6784282277466	
F1 Score Class 0	25.538707102952912	
F1 Score Class 1	38.961038961038966	
F1 Score Class 2	58.53471423091676	
F1 Score Class 3	47.13142857142857	

test_multiclasse_focal

Metric	Value	
Test Accuracy	54.44733160103937	
Macro Precision	54.49979750696173	
Macro Recall	52.30618703407337	
Macro F1 Score	52.50060102933087	
Top-2 Accuracy	74.49530281830901	
Accuracy Class 0	42.06680584551148	
Accuracy Class 1	43.26171875	
Accuracy Class 2	63.19052987598647	
Accuracy Class 3	60.70569366479551	
Precision Class 0	45.63986409966025	
Precision Class 1	46.87830687830688	
Precision Class 2	84.53996983408749	
Precision Class 3	40.94104921579232	
Recall Class 0	42.06680584551148	
Recall Class 1	43.26171875	
Recall Class 2	63.19052987598647	
Recall Class 3	60.70569366479551	
F1 Score Class 0	43.780554046713746	
F1 Score Class 1	44.99746063991875	
F1 Score Class 2	72.3225806451613	
F1 Score Class 3	48.90180878552972	

CNN - 3 convolutional layers

test_binario_crossentropy

Metric	Value
Test Accuracy	81.19827871565707
Macro Precision	83.46100240956918
Macro Recall	83.62172935450945
Macro F1 Score	81.1973701512163
Accuracy Class 0	69.72942502818489
Accuracy Class 1	97.514033680834
Precision Class 0	97.55520504731862
Precision Class 1	69.36679977181974
Recall Class 0	69.72942502818489
Recall Class 1	97.514033680834
F1 Score Class 0	81.32807363576595
F1 Score Class 1	81.06666666666666

test_binario_focal

Metric	Value
Test Accuracy	86.26282687851705
Macro Precision	86.12223746070654
Macro Recall	85.38499162363968
Macro F1 Score	85.69477741821493
Accuracy Class 0	90.41713641488163
Accuracy Class 1	80.35284683239776
Precision Class 0	86.74959437533802
Precision Class 1	85.49488054607508
Recall Class 0	90.41713641488163
Recall Class 1	80.35284683239774
F1 Score Class 0	88.54540436102677
F1 Score Class 1	82.84415047540307

test_multiclasse_crossentropy

Metric	Value
Test Accuracy	50.48970617629422
Macro Precision	58.03457858241728
Macro Recall	48.751304915939855
Macro F1 Score	48.3868999766257
Top-2 Accuracy	76.3541874875075
Accuracy Class 0	34.34237995824635
Accuracy Class 1	30.17578125
Accuracy Class 2	49.49267192784667
Accuracy Class 3	80.99438652766639
Precision Class 0	52.05696202531646
Precision Class 1	48.58490566037736
Precision Class 2	95.64270152505446
Precision Class 3	35.85374511892084
Recall Class 0	34.34237995824635
Recall Class 1	30.17578125
Recall Class 2	49.49267192784668
Recall Class 3	80.99438652766639
F1 Score Class 0	41.38364779874213
F1 Score Class 1	37.2289156626506
F1 Score Class 2	65.23031203566121
F1 Score Class 3	49.70472440944882

test_multiclasse_focal

Metric	Value
Test Accuracy	58.7847291625025
Macro Precision	58.51850390495921
Macro Recall	55.7842396085244
Macro F1 Score	55.79464249373178
Top-2 Accuracy	79.75214871077354
Accuracy Class 0	49.89561586638831
Accuracy Class 1	33.984375
Accuracy Class 2	71.815107102593
Accuracy Class 3	67.44186046511628
Precision Class 0	50.31578947368421
Precision Class 1	52.567975830815705
Precision Class 2	87.86206896551724
Precision Class 3	43.32818134981969
Recall Class 0	49.89561586638831
Recall Class 1	33.984375
Recall Class 2	71.81510710259302
Recall Class 3	67.44186046511628
F1 Score Class 0	50.10482180293501
F1 Score Class 1	41.281138790035584
F1 Score Class 2	79.03225806451613
F1 Score Class 3	52.76035131744041

CNN - 4 convolutional layers

test_binario_crossentropy

Metric	Value
Test Accuracy	86.36213174445548
Macro Precision	86.70963563264694
Macro Recall	87.81607989953793
Macro F1 Score	86.29527676966985
Accuracy Class 0	79.48139797068771
Accuracy Class 1	96.15076182838813
Precision Class 0	96.70781893004116
Precision Class 1	76.71145233525272
Recall Class 0	79.48139797068771
Recall Class 1	96.15076182838813
F1 Score Class 0	87.25247524752476
F1 Score Class 1	85.33807829181495

test_binario_focal

Metric	Value
Test Accuracy	86.62694472029129
Macro Precision	86.51019135978902
Macro Recall	87.64848488684002
Macro F1 Score	86.5068562210123
Accuracy Class 0	81.79255918827508
Accuracy Class 1	93.50441058540497
Precision Class 0	94.71279373368147
Precision Class 1	78.30758898589657
Recall Class 0	81.79255918827508
Recall Class 1	93.50441058540497
F1 Score Class 0	87.77979431336964
F1 Score Class 1	85.23391812865498

test_multiclasse_crossentropy

Metric	Value
Test Accuracy	57.445532680391764
Macro Precision	60.44842199720825
Macro Recall	52.03492222320501
Macro F1 Score	49.80556981419406
Top-2 Accuracy	80.7115730561663
Accuracy Class 0	31.106471816283925
Accuracy Class 1	13.57421875
Accuracy Class 2	75.64825253664036
Accuracy Class 3	87.81074578989575
Precision Class 0	59.24453280318092
Precision Class 1	53.66795366795367
Precision Class 2	88.75661375661376
Precision Class 3	40.12458776108465
Recall Class 0	31.106471816283925
Recall Class 1	13.57421875
Recall Class 2	75.64825253664036
Recall Class 3	87.81074578989575
F1 Score Class 0	40.79397672826831
F1 Score Class 1	21.66796570537802
F1 Score Class 2	81.67985392574559
F1 Score Class 3	55.08048289738431

test_multiclasse_focal

Metric	Value
Test Accuracy	62.54247451529083
Macro Precision	60.67984593380092
Macro Recall	58.76953721604187
Macro F1 Score	59.006072533856795
Top-2 Accuracy	82.71037377573455
Accuracy Class 0	42.48434237995825
Accuracy Class 1	45.5078125
Accuracy Class 2	80.04509582863585
Accuracy Class 3	67.04089815557337
Precision Class 0	57.002801120448176
Precision Class 1	50.70729053318824
Precision Class 2	86.79706601466992
Precision Class 3	48.212226066897344
Recall Class 0	42.48434237995824
Recall Class 1	45.5078125
Recall Class 2	80.04509582863585
Recall Class 3	67.04089815557339
F1 Score Class 0	48.68421052631579
F1 Score Class 1	47.96706124549665
F1 Score Class 2	83.28445747800586
F1 Score Class 3	56.088560885608864

EfficientNet

test_binario_crossentropy

Metric	Value
Test Accuracy	76.13373055279709
Macro Precision	75.64624528910244
Macro Recall	76.31969488892847
Macro F1 Score	75.77027570150689
Accuracy Class 0	75.25366403607666
Accuracy Class 1	77.38572574178028
Precision Class 0	82.56029684601113
Precision Class 1	68.73219373219374
Recall Class 0	75.25366403607666
Recall Class 1	77.38572574178028
F1 Score Class 0	78.73783544677086
F1 Score Class 1	72.80271595624292

test_binario_focal

Metric	Value
Test Accuracy	76.36544190665343
Macro Precision	75.64224040446783
Macro Recall	75.42114603797705
Macro F1 Score	75.52130571556972
Accuracy Class 0	80.83427282976325
Accuracy Class 1	70.00801924619086
Precision Class 0	79.3141592920354
Precision Class 1	71.97032151690024
Recall Class 0	80.83427282976324
Recall Class 1	70.00801924619086
F1 Score Class 0	80.06700167504188
F1 Score Class 1	70.97560975609755

test_multiclasse_crossentropy

Metric	Value
Test Accuracy	34.539276434139516
Macro Precision	44.253621483037044
Macro Recall	37.736038445800745
Macro F1 Score	34.05623891110632
Top-2 Accuracy	57.84529282430542
Accuracy Class 0	71.08559498956159
Accuracy Class 1	35.9375
Accuracy Class 2	24.91544532130778
Accuracy Class 3	19.0056134723336
Precision Class 0	23.645833333333332
Precision Class 1	40.04352557127312
Precision Class 2	74.53625632377741
Precision Class 3	38.788870703764324
Recall Class 0	71.08559498956159
Recall Class 1	35.9375
Recall Class 2	24.91544532130778
Recall Class 3	19.0056134723336
F1 Score Class 0	35.4872329338197
F1 Score Class 1	37.879567678847145
F1 Score Class 2	37.34685255597803
F1 Score Class 3	25.511302475780408

test_multiclasse_focal

Metric	Value
Test Accuracy	43.713771736957824
Macro Precision	44.8270837336304
Macro Recall	42.8194715194782
Macro F1 Score	40.34815986293057
Top-2 Accuracy	66.0203877673396
Accuracy Class 0	56.68058455114823
Accuracy Class 1	41.2109375
Accuracy Class 2	58.22998872604284
Accuracy Class 3	15.156375300721733
Precision Class 0	30.066445182724255
Precision Class 1	34.93377483443709
Precision Class 2	64.04215747055176
Precision Class 3	50.26595744680851
Recall Class 0	56.68058455114823
Recall Class 1	41.2109375
Recall Class 2	58.22998872604283
Recall Class 3	15.156375300721733
F1 Score Class 0	39.29088277858177
F1 Score Class 1	37.81362007168458
F1 Score Class 2	60.99793327428403
F1 Score Class 3	23.290203327171906

MobileNet

test_binario_crossentropy

Metric	Value
Test Accuracy	83.41608738828202
Macro Precision	84.33986310506924
Macro Recall	85.18851105109988
Macro F1 Score	83.3793656836927
Accuracy Class 0	75.0281848928974
Accuracy Class 1	95.34883720930233
Precision Class 0	95.82433405327573
Precision Class 1	72.85539215686273
Recall Class 0	75.02818489289741
Recall Class 1	95.34883720930233
F1 Score Class 0	84.1606070186532
F1 Score Class 1	82.5981243487322

test_binario_focal

Metric	Value
Test Accuracy	86.82555445216816
Macro Precision	86.3094913931715
Macro Recall	86.76939649521874
Macro F1 Score	86.50302547541904
Accuracy Class 0	87.0913190529876
Accuracy Class 1	86.44747393744989
Precision Class 0	90.14002333722287
Precision Class 1	82.47895944912013
Recall Class 0	87.0913190529876
Recall Class 1	86.44747393744989
F1 Score Class 0	88.58944954128441
F1 Score Class 1	84.41660140955365

test_multiclasse_crossentropy

Metric	Value	
Test Accuracy	58.524885068958625	
Macro Precision	55.82503122554371	
Macro Recall	55.56120068156907	
Macro F1 Score	54.9286485554218	
Top-2 Accuracy	78.9926044373376	
Accuracy Class 0	61.06471816283925	
Accuracy Class 1	38.8671875	
Accuracy Class 2	79.65050732807215	
Accuracy Class 3	42.662389735364876	
Precision Class 0	41.19718309859155	
Precision Class 1	50.1891551071879	
Precision Class 2	80.51282051282051	
Precision Class 3	51.40096618357488	
Recall Class 0	61.06471816283925	
Recall Class 1	38.8671875	
Recall Class 2	79.65050732807215	
Recall Class 3	42.662389735364876	
F1 Score Class 0	49.201009251471824	
F1 Score Class 1	43.80847550908091	
F1 Score Class 2	80.07934258996883	
F1 Score Class 3	46.625766871165645	

test_multiclasse_focal

Metric	Value	
Test Accuracy	59.34439336398161	
Macro Precision	58.47301983373478	
Macro Recall	56.952907552651354	
Macro F1 Score	56.12510313036013	
Top-2 Accuracy	80.63162102738357	
Accuracy Class 0	67.22338204592901	
Accuracy Class 1	33.10546875	
Accuracy Class 2	75.19729425028184	
Accuracy Class 3	52.285485164394544	
Precision Class 0	40.42686754551161	
Precision Class 1	54.153354632587856	
Precision Class 2	87.64783180026281	
Precision Class 3	51.66402535657686	
Recall Class 0	67.22338204592901	
Recall Class 1	33.10546875	
Recall Class 2	75.19729425028186	
Recall Class 3	52.28548516439455	
F1 Score Class 0	50.490003920031356	
F1 Score Class 1	41.09090909090909	
F1 Score Class 2	80.94660194174756	
F1 Score Class 3	51.972897568752494	

ResNet - training for 5 epochs

test_binario_crossentropy

Metric	Value
Test Accuracy	87.45448526977822
Macro Precision	87.38486509042042
Macro Recall	88.55559995624222
Macro F1 Score	87.3488897838903
Accuracy Class 0	82.24351747463359
Accuracy Class 1	94.86768243785085
Precision Class 0	95.79776756401839
Precision Class 1	78.97196261682244
Recall Class 0	82.24351747463359
Recall Class 1	94.86768243785085
F1 Score Class 0	88.50470124355473
F1 Score Class 1	86.19307832422587

test_binario_focal

Metric	Value
Test Accuracy	85.63389606090699
Macro Precision	85.7588583151776
Macro Recall	86.87440612825912
Macro F1 Score	85.54034059628171
Accuracy Class 0	79.76324689966178
Accuracy Class 1	93.98556535685645
Precision Class 0	94.96644295302013
Precision Class 1	76.55127367733508
Recall Class 0	79.76324689966178
Recall Class 1	93.98556535685645
F1 Score Class 0	86.703431372549
F1 Score Class 1	84.37724982001441

test_multiclasse_crossentropy

Metric	Value	
Test Accuracy	51.72896262242654	
Macro Precision	59.98067506157385	
Macro Recall	51.81059804703082	
Macro F1 Score	48.4872659222809	
Top-2 Accuracy	76.43413951629023	
Accuracy Class 0	91.23173277661796	
Accuracy Class 1	31.4453125	
Accuracy Class 2	64.03607666290868	
Accuracy Class 3	20.52927024859663	
Precision Class 0	31.270125223613597	
Precision Class 1	57.29537366548043	
Precision Class 2	95.94594594594594	
Precision Class 3	55.41125541125541	
Recall Class 0	91.23173277661796	
Recall Class 1	31.4453125	
Recall Class 2	64.03607666290868	
Recall Class 3	20.52927024859663	
F1 Score Class 0	46.57607247535306	
F1 Score Class 1	40.605296343001264	
F1 Score Class 2	76.80865449628126	
F1 Score Class 3	29.959040374488005	

test_multiclasse_focal

Metric	Value	
Test Accuracy	59.26444133519888	
Macro Precision	60.72658784268653	
Macro Recall	57.88515218430348	
Macro F1 Score	56.74230647273768	
Top-2 Accuracy	80.9314411353188	
Accuracy Class 0	76.7223382045929	
Accuracy Class 1	38.37890625	
Accuracy Class 2	73.05524239007892	
Accuracy Class 3	43.3841218925421	
Precision Class 0	38.004136504653566	
Precision Class 1	57.45614035087719	
Precision Class 2	92.24199288256227	
Precision Class 3	55.20408163265306	
Recall Class 0	76.7223382045929	
Recall Class 1	38.37890625	
Recall Class 2	73.05524239007892	
Recall Class 3	43.3841218925421	
F1 Score Class 0	50.829875518672196	
F1 Score Class 1	46.01873536299766	
F1 Score Class 2	81.53507392261717	
F1 Score Class 3	48.585541086663675	

ResNet - training for 10 epochs

test_binario_crossentropy

Metric	Value
Test Accuracy	88.18272095332671
Macro Precision	87.69131490504162
Macro Recall	88.5086326687997
Macro F1 Score	87.96097402409664
Accuracy Class 0	86.64036076662909
Accuracy Class 1	90.37690457097032
Precision Class 0	92.75799637899819
Precision Class 1	82.62463343108504
Recall Class 0	86.64036076662909
Recall Class 1	90.37690457097032
F1 Score Class 0	89.59487030020401
F1 Score Class 1	86.32707774798928

test_binario_focal

Metric	Value
Test Accuracy	84.50844091360477
Macro Precision	84.88350250041916
Macro Recall	85.92803110780417
Macro F1 Score	84.4353643360468
Accuracy Class 0	77.7903043968433
Accuracy Class 1	94.06575781876504
Precision Class 0	94.91059147180194
Precision Class 1	74.85641352903637
Recall Class 0	77.7903043968433
Recall Class 1	94.06575781876504
F1 Score Class 0	85.50185873605949
F1 Score Class 1	83.3688699360341

test_multiclasse_crossentropy

Metric	Value
Test Accuracy	59.304417349590246
Macro Precision	61.62194260656547
Macro Recall	56.93016056115607
Macro F1 Score	54.78496168940568
Top-2 Accuracy	80.19188486907855
Accuracy Class 0	81.00208768267224
Accuracy Class 1	32.421875
Accuracy Class 2	82.2998872604284
Accuracy Class 3	31.996792301523655
Precision Class 0	36.74242424242424
Precision Class 1	62.99810246679317
Precision Class 2	84.98253783469151
Precision Class 3	61.76470588235294
Recall Class 0	81.00208768267223
Recall Class 1	32.421875
Recall Class 2	82.2998872604284
Recall Class 3	31.99679230152366
F1 Score Class 0	50.553745928338756
F1 Score Class 1	42.81108961960026
F1 Score Class 2	83.6197021764032
F1 Score Class 3	42.15530903328051

test_multiclasse_focal

Metric	Value
Test Accuracy	60.18388966620028
Macro Precision	59.26289745282824
Macro Recall	58.43937236776649
Macro F1 Score	57.60737034497505
Top-2 Accuracy	79.97201678992604
Accuracy Class 0	67.5365344467641
Accuracy Class 1	48.046875
Accuracy Class 2	75.59188275084554
Accuracy Class 3	42.582197273456295
Precision Class 0	40.949367088607595
Precision Class 1	49.596774193548384
Precision Class 2	88.28176431863068
Precision Class 3	58.223684210526315
Recall Class 0	67.5365344467641
Recall Class 1	48.046875
Recall Class 2	75.59188275084556
Recall Class 3	42.582197273456295
F1 Score Class 0	50.985027580772254
F1 Score Class 1	48.80952380952381
F1 Score Class 2	81.44549043425448
F1 Score Class 3	49.18943955534969

Confusion matrices

CNN2conv - test binario cross entropy		
	Pred 0	Pred 1
True 0	1331	443
True 1	154	1093

CNN2conv - test multiclasse cross entropy				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	160	120	12	666
True 1	64	345	29	586
True 2	31	135	763	845
True 3	40	147	29	1031

CNN2conv - test binario focal loss		
	Pred 0	Pred 1
True 0	1345	429
True 1	168	1079

CNN2conv - test multiclasse focal loss				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	403	160	62	333
True 1	149	443	72	360
True 2	139	115	1121	399
True 3	192	227	71	757

CNN3conv - test binario cross entropy		
	Pred 0	Pred 1
True 0	1237	537
True 1	31	1216

CNN3conv - test multiclasse cross entropy				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	329	99	13	517
True 1	103	309	12	600
True 2	74	132	878	690
True 3	126	96	15	1010

CNN3conv - test binario focal loss		
	Pred 0	Pred 1
True 0	1604	170
True 1	245	1002

CNN3conv - test multiclasse focal loss				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	478	89	48	343
True 1	168	348	63	445
True 2	115	73	1274	312
True 3	189	152	65	841

CNN4conv - test binario cross entropy		
	Pred 0	Pred 1
True 0	1410	364
True 1	48	1199

CNNconv4 - test multiclasse cross entropy				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	298	53	48	559
True 1	115	139	65	705
True 2	26	36	1342	370
True 3	64	31	57	1095

CNN4conv - test binario focal loss		
	Pred 0	Pred 1
True 0	1451	323
True 1	81	1166

CNN4conv - test multiclasse focal loss				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	407	159	64	328
True 1	103	466	64	391
True 2	82	93	1420	179
True 3	122	201	88	836

EfficientNet - test binario cross entropy		
	Pred 0	Pred 1
True 0	1335	439
True 1	282	965

EfficientNet - test binario focal loss		
	Pred 0	Pred 1
True 0	1434	340
True 1	374	873

EfficientNet - test multiclasse cross entropy				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	681	156	41	80
True 1	491	368	47	118
True 2	950	206	442	176
True 3	758	189	63	237

EfficientNet - test multiclasse focal loss				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	543	205	163	47
True 1	328	422	200	74
True 2	418	257	1033	66
True 3	517	324	217	189

MobileNet - test binario cross entropy		
	Pred 0	Pred 1
True 0	1331	443
True 1	58	1189

MobileNet - test binario focal loss		
	Pred 0	Pred 1
True 0	1545	229
True 1	169	1078

MobileNet - test multiclasse cross entropy				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	585	106	89	178
True 1	284	398	110	232
True 2	189	79	1413	93
True 3	362	210	143	532

MobileNet - test multiclasse focal loss				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	644	74	56	184
True 1	324	339	52	309
True 2	251	72	1334	117
True 3	374	141	80	652

ResNet - test binario cross entropy		
	Pred 0	Pred 1
True 0	1459	315
True 1	64	1183

ResNet - test binario focal loss		
	Pred 0	Pred 1
True 0	1415	359
True 1	75	1172

ResNet - test multiclasse cross entropy				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	874	44	11	29
True 1	594	322	15	93
True 2	505	49	1136	84
True 3	822	147	22	256

ResNet - test multiclasse focal loss				
	Pred 0	Pred 1	Pred 2	Pred 3
True 0	735	75	41	107
True 1	402	393	31	198
True 2	297	47	1296	134
True 3	500	169	37	541

Comment on the results

All the tests show good results in the **binary task**, with an acceptable balance between classes in most cases. Using **focal loss** in the computation led to little to no general improvement in this task – with an exception for the CNN with 3 convolutional layers, where the improvement is more significant – while for the multiclass task it brought some improvement in all tests, a significant improvement in some cases with respect to the test accuracy.

Using focal loss on a balanced dataset for a classification task may seem counterintuitive, as focal loss is primarily designed to handle imbalanced datasets or class imbalance problems. However, it can still be useful because it reduces the importance of well-classified samples (i.e., those with high predicted probabilities), focusing more on hard or uncertain samples.

In a balanced dataset, models tend to quickly converge on easier examples, ignoring the more challenging ones. This can lead to metrics like precision and recall becoming unbalanced, which can be addressed by focal loss. It forces the model to focus more on difficult samples, increasing the chances of improving the balance between precision and recall.

However, in the case of the ResNet model, a performance decline is observed when using focal loss in the binary case. This likely occurs because, even with a standard loss function like cross-entropy, the metrics were already well balanced. This effect depends on how focal loss adjusts the weights of harder samples: it amplifies the weight of examples classified with greater uncertainty, which can lead to overfitting on samples that are considered difficult to classify for the model. So, it may excessively penalize correctly classified samples that were already well-predicted and negatively affect the trade-off between false positives and false negatives for all classes in general.

In fact, the cases when the focal loss lead to a significant improvement are the ones where it is applied to perform the multiclass classification task using the custom CNNs and EfficientNet: the same tests with the cross-entropy loss showed a bigger gap between accuracy, precision and recall among certain classes, so applying focal loss led to a more significant test accuracy improvement and better balance between the other values.

Regarding the **multiclass task**, a significant drop in overall performance is observed in comparison to the binary task. A pattern can be identified in the results: the “happy” class tends to achieve the best performance on average and seems to be the easiest one to classify.

This drop in performance may be due to various factors:

- The classes “angry,” “fear,” and “sad” have more overlapping features and may be difficult to classify correctly, even though weights to make classification easier for these classes were passed to the loss functions. Even human testing of image classification revealed some difficulties in distinguishing clearly between these classes.
- The dataset may be still too small to allow pre-trained to train effectively, avoid overfitting and generalize properly, even though versions of the models with relatively fewer parameters were chosen and techniques like weight decay were applied to the optimizer. However, this issue may be secondary to the overlapping features in the case of the multiclass task, as the performance in the binary task, where the dataset is even smaller, does not show such significant problems.
- The dataset may be too different from the domain in which the pre-trained models were trained (generic image classification), to the point where the models might struggle to extract facial emotion features and fail using all of its capabilities. The decision to train not only the last layer but the last few layers was meant to mitigate this potential issue while still retaining the benefits of transfer learning.

Actually, the last two points of the bulleted list could be problems that affect both tasks: this could explain why models deeper than our custom CNNs unexpectedly do not achieve significantly better performances throughout the whole project.

The fact that the conditions to fully leverage the pre-trained models' potential are not met is also evident when looking at the validation results of the ResNet model trained for 10 epochs. After around the fifth epoch, the model shows clear signs of overfitting if no technique is applied to prevent it. In some cases, the validation accuracy is far from the training accuracy value, and while the training accuracy continues to increase, the validation accuracy stabilizes. So, the model fails to generalize properly according to its capabilities.

validation_multiclasse_focal

Epoch	Train Loss	Train Accuracy	Validation Accuracy
1	0.6612050279974937	56.752110034385744	61.331666145670525
2	0.4193992327153683	66.09878086902157	59.1434823382307
3	0.3149767840653658	71.30353235386058	63.801187871209756
4	0.20681137705221772	77.47733666770866	62.48827758674586
5	0.10688004518859089	84.42482025633011	65.14535792435136
6	0.036294346472714095	91.23944982807127	68.80275085964364
7	0.0038569496004492976	97.91341044076273	68.89653016567678
8	0.000602162730768896	99.3826195686152	69.11534854642076
9	0.00020110083466875038	99.8046264457643	69.17786808377618
10	0.00015734322795310618	99.78899656142545	69.2403876211316

Overfitting example

With the usage of weight decay, the overfitting issue is mitigated, but the model still fails to generalize:

validation_multiclasse_focal

Epoch	Train Loss	Train Accuracy	Validation Accuracy
1	0.8359095367789269	49.230348491951865	50.132833255196125
2	0.707908246256411	53.93420847007345	55.25863416158775
3	0.626574503928423	57.20034380371933	57.80590717299578
4	0.5862577389366925	59.36474449132677	58.665416471323645
5	0.5551923654600978	60.8923269260822	57.712142522269104
6	0.5381091963872313	61.4588216908892	58.3216127519925
7	0.5191894488967955	62.00578215346148	58.337240193780275
8	0.5105632155202329	62.94342866072824	59.74370995468042
9	0.4963562122359872	63.54899203000469	56.61822159712455
10	0.48547005048021674	63.9943741209564	59.10298484138146

Example after applying weight decay

Looking at the train loss, seems that the model keeps learning until the last epoch and there are not clear signs of overfitting so, in order to verify whether the issue with the poor

performances depend on the number of training epochs or not at this point, a final test with the validation set is done by training the regularized ResNet model for 20 epochs:

validation_multiclasse_focal

Epoch	Train Loss	Train Accuracy	Validation Accuracy
1	0.8359095367789269	49.230348491951865	50.132833255196125
2	0.707908246256411	53.93420847007345	55.25863416158775
3	0.626574503928423	57.20034380371933	57.80590717299578
4	0.5862577389366925	59.36474449132677	58.665416471323645
5	0.5551923654600978	60.8923269260822	57.712142522269104
6	0.5381091963872313	61.4588216908892	58.3216127519925
7	0.5191894488967955	62.00578215346148	58.337240193780275
8	0.5105632155202329	62.94342866072824	59.74370995468042
9	0.4963562122359872	63.54899203000469	56.61822159712455
10	0.48547005048021674	63.9943741209564	59.10298484138146
11	0.4835577052459121	64.11939365525863	54.75855602437881
12	0.4680090628750622	64.9359274886701	56.07126113455227
13	0.47153033040463926	64.52961400218784	57.82153461478356
14	0.47135906118899584	64.517893420847	59.05610251601813
15	0.4632397530786693	65.00234411626816	58.83731833098922
16	0.46223492015153167	65.3617752773871	59.55618065322707
17	0.45631043244153263	65.56102516018127	60.02500390686045
18	0.45250384501181545	65.41647132364432	55.993123925613375
19	0.4521760086528957	65.50632911392405	58.602906704172526
20	0.4474299655668437	65.8852945772777	55.711829973433346

Example with 20 epochs training

The model seems to slowly keep learning but still fails to generalize: train accuracy becomes quite stagnant by epoch 15 and the validation accuracy shows clear sign of overfitting by epoch 18, with no improvement in terms of accuracy reached in any epoch with respect to validation and testing after training for 5 or 10 epochs. So, the model still fails to learn properly and generalize in the same way after applying regularization and increasing the number of epochs, showing that the issue may rely on the characteristics of the dataset even though data augmentation techniques have been applied. Unfreezing more layers during training could mitigate this problem, but at the same time it could encourage overfitting since the model would be losing more general knowledge obtained from the pre-training phase and it could start focusing too much on specific features of the specific task, especially in this case where the dataset is still relatively small. Probably, the best option would be switching to bigger pre-trained models, that could have more capacity to catch details in the images and classify accordingly in a more precise way.

Below are examples of how the images from the more difficult-to-classify classes can be quite tricky, in the multiclass task. Take into account the following two images depicting men:



These two images both come from the *angry* training set, but they can be misleading. They can be mistaken for expressing fear in the first one and sadness in the second.

Another example is given by the following pictures of a toddler and a baby:



Both pictures can be interpreted as showing the same emotion, but the first one comes from the *sad* class, while the second one comes from the *fear* class.

As the images contained in the dataset are not very clear, all the tests were repeated after applying a median filter to the images to denoise them, but this did not lead to a significant change in the results of the test phase for the classification task.

However, to address this ambiguity throughout the dataset and test the models' abilities in a proper way, also another metric has been taken into account in the case of the multiclass task: **top-2 score**, which measures the percentage of times the model's correct prediction is among the two highest-probability classes assigned by the model. As can be seen from the results, the top-2 score value is always high enough to assess that models are performing in an acceptable way despite the overlapping features, making this another proof that the issues that lead to a poor performance may be related to the dataset's characteristics.

Looking at the performances of the pre-trained models only, ResNet performs better on average in this case, probably due to its deeper architecture and the use of residual connections, which allow it to handle complex features more effectively especially with relatively small datasets that have subtle differences between the classes. EfficientNet is optimized for efficiency so, while the resources are used in a more optimized way for a faster convergence, it may struggle with capturing fine-grained details on smaller datasets or datasets with complex features. MobileNet is lightweight and fast, but sacrifices some accuracy in comparison with ResNet, making it less suitable for complex tasks like facial emotion recognition, so this may be the reason why its performances don't reach those of the ResNet model on average.

However, MobileNet seems to perform better than EfficientNet, which could have several reasons:

- MobileNet might perform better on smaller datasets compared to EfficientNet, as it has fewer parameters and a simpler structure. EfficientNet, being more optimized for large datasets and complex tasks with limited computational resources, may struggle to generalize well when the data is limited.
- MobileNet's architecture tends to converge faster with smaller datasets, thanks to its simpler architecture. Within a few epochs of training, MobileNet may be able to learn faster and more effectively compared to EfficientNet.

For what concerns the performances of the custom CNNs in general, they meet the expectations: as the convolutional layers increase, the performance becomes more promising overall, with the focal loss that helps boosting these performances whenever metrics are most unbalanced. Having more convolutional layers means being able to extract more features and more complex features from the images, therefore the behavior of the CNNs is in line with what was expected to happen during the testing phase.